AE 601: Midterm Project

Due on Wednesday, November 4, 2026

Jacobs 4:00pm

Hannah Douthit

Contents

Summary **3**

Problem **3**

Theory and Methodology **3**
Nondimensionalize . . . 3
Analytical Solution . . . 4
Numerical Solutions . . . 6
Explicit Euler . . . 6
Implicit Euler . . . 6
Fourth-Order Runge-Kutta . . . 7
Error and Convergence . . . 7

Results and Discussion **8**

Conclusion **14**

Handwritten Derivations **16**
Nondimensionalization . . . 16
Analytical Solution . . . 17
Explicit Euler Method . . . 22
Implicit Euler . . . 22

MATLAB Code: Initial w/ some errors **24**

Final Matlab Code **29**

Summary

The flow between two parallel plates initially at rest was investigated using explicit Euler, implicit Euler, and fourth-order Runge-Kutta methods. The numerical solutions for each of these methods were compared to a derived analytical solution using the velocity profile at τ_{final}, flow rate over time, and the wall shear stress over time. Grid refinement in all numerical methods resulted in the reduction of error, with RK4 producing the most accurate velocity profile in comparison to the Euler methods. A diffusion-number study on the numerical methods demonstrated that the reduction in τ and its associated total error was often limited by the grid spacing. Overall, all methods on finer grids and smaller time steps agreed with the derived analytical solution. The study demonstrated the importance of balancing and selecting spatial and temporal resolutions.

Problem

This project aims to investigate the fluid flow over time for a fluid initially at rest between two stationary, parallel plates separated by distance $2h$. The objective is to nondimensionalize a simplified Navier-Stokes equation and its associated boundary and initial conditions, derive an exact solution for the velocity field, and obtain a numerical solution using explicit and implicit time methods. The numerical solutions are to be compared to exact solutions to investigate the accuracy and stability. In addition, a fourth-order Runge-Kutta study will be conducted to examine the effect of ΔT by running simulations with differing time steps.

Theory and Methodology

The x component of the Navier-Stokes Equation is defined as

$$\rho\left(\frac{\partial u}{\partial t}+u\frac{\partial u}{\partial x}+v\frac{\partial u}{\partial y}+w\frac{\partial u}{\partial z}\right)=-\frac{\partial p}{\partial x}+\mu\left(\frac{\partial^2 u}{\partial x^2}+\frac{\partial^2 u}{\partial y^2}+\frac{\partial^2 u}{\partial z^2}\right). \tag{1}$$

The flow can be simplified by assuming that the flow field is unidirectional, fully developed in the x direction, and is driven by a constant pressure gradient. The fluid is initially at rest. The resulting equations is as follows

$$\frac{\partial u}{\partial t}=-\frac{1}{\rho}\frac{dp}{dx}+\nu\frac{\partial^2 u}{\partial y^2} \tag{2}$$

The boundary conditions can be defined by the flow moving through two parallel plates through the no-slip conditions. This results in the following conditions.

$$u(-h,t)=0, \tag{3}$$
$$u(h,t)=0. \tag{4}$$

Because the fluid through the channel is initially at rest, an initial condition can be defined as

$$u(y,0)=0. \tag{5}$$

Nondimensionalize

Scaling was utilized to nondimensionalize the governing equations. A characteristic length was selected as the half distance h between the two plates. The dimensionless velocity, spacial coordinate y, and time are defined by

$$U = \frac{u}{U_\infty}, \qquad Y = \frac{y}{h}, \qquad \tau = \frac{\nu t}{h^2}. \tag{6}$$

The pressure gradient term is then represented by

$$G = -\frac{1}{\rho}\frac{dp}{dx}. \tag{7}$$

A characteristic velocity was chosen to be

$$U_\infty = \frac{Gh^2}{\nu}. \tag{8}$$

which allowed for the simplified Navier-Stokes equation to be nondimensionalized as follows

$$\frac{\partial U}{\partial \tau} = 1 + \frac{\partial^2 U}{\partial Y^2} \tag{9}$$

In the same manner, the boundary and initial conditions can be rewritten to incorporate the scaled velocity, spacial coordinates, and time

$$U(Y, 0) = 0 \tag{10}$$

and

$$U(-1, \tau) = 0, \qquad U(1, \tau) = 0. \tag{11}$$

Analytical Solution

In order to compare the numerical solutions and consider the effects of Δt and Δy, an analytical solution must be derived for the transient velocity field. This solution will provide as a reference to evaluate the accuracy of the explicit Euler, implicit Euler, and 4th order Runge Kutta methods.

To evaluate the analytical solution to obtain the velocity field, the equation must be decomposed into its steady state and transient components as follows

$$U(Y, \tau) = U_s(Y) + V(Y, \tau). \tag{12}$$

For the steady state solution, the governing equation becomes

$$0 = 1 + \frac{d^2 U_s}{dY^2} \tag{13}$$

and is integrated twice to obtain the following equation with constants

$$\begin{aligned} \frac{dU_s}{dY} &= -Y + C_1, \\ U_s(Y) &= -\frac{Y^2}{2} + C_1 Y + C_2. \end{aligned}$$

Applying the boundary conditions $U(-1) = U(1) = 0$ gives us the steady state solution for the velocity field

$$U_s(Y) = \frac{1}{2}(1 - Y^2) \tag{14}$$

When solving for the transient solution, the governing equation can be found through substituting $U(Y, \tau) = U_S(Y) + V(Y, \tau)$ into Equation (9) to obtain

$$\frac{\partial V}{\partial \tau} = \frac{\partial^2 V}{\partial Y^2} \tag{15}$$

Separation of variables is used to solve for $V(Y,\tau)$ to get the following components

$$V(Y,\tau) = F(Y)T(\tau). \tag{16}$$

Substituting the expression into the transient governing equation gives

$$F(Y)\frac{dT}{d\tau} = T(\tau)\frac{d^2F}{dY^2} \tag{17}$$

which is then divided through by $F(Y)T(\tau)$ and is sent equal to a separation constant $-\lambda$. This creates two ODEs, one being a temporal equation and the other being spatial.

$$\frac{dT}{d\tau} + \lambda T = 0,$$
$$\frac{d^2F}{dY^2} + \lambda F = 0.$$

For the temporal equation, the solution is

$$T(\tau) = e^{-\lambda\tau} \tag{18}$$

and the spatial equation solution is

$$F(Y) = A\cos(kY) + B\sin(kY) \tag{19}$$

$$\lambda = k^2 \tag{20}$$

In order to find the exact solution, boundary conditions are applied to the spatial solution being $F(1) = F(-1) = 0$. The initial transient condition is noted to be symmetric about $Y = 0$, only cosine modes are needed to define the solution. Applying the no-slip conditions give

$$\cos(k) = 0 \tag{21}$$

which allows for eigenvalues of

$$k = \frac{(2n+1)\pi}{2}, \qquad n = 0, 1, 2, \ldots \tag{22}$$

Because $\lambda = k^2$, this value can be used to obtain the transient solution with regards to both the temporal and spatial equations that make up $V(Y,\tau)$.

$$V(Y,\tau) = \sum_{n=0}^{\infty} A_n \cos\left(\frac{(2n+1)\pi Y}{2}\right) e^{-(2n+1)^2\pi^2\tau/4} \tag{23}$$

To find the exact solution, the coefficient A_n must be obtained. To do so, the initial transient condition was expanded as a Fourier cosine series. The orthogonality of the eigenfunctions was used to isolate each coefficient and an integral was taken across the flow channel from $-1 < Y < 1$ to obtain A_n.

$$A_n = \int_{-1}^{1} V(Y,0)\cos\left(\frac{(2n+1)\pi Y}{2}\right) dY. \tag{24}$$

The integral resulted in A_n being

$$A_n = -\frac{16(-1)^n}{(2n+1)^3\pi^3}. \tag{25}$$

The exact analytical solution is now obtained through combining both the steady state and transient solutions.

$$\boxed{U(Y,\tau) = \frac{1}{2}(1-Y^2) - \frac{16}{\pi^3}\sum_{n=0}^{\infty}\frac{(-1)^n}{(2n+1)^3}\cos\left(\frac{(2n+1)\pi Y}{2}\right)e^{-(2n+1)^2\pi^2\tau/4}}. \tag{26}$$

Numerical Solutions

Explicit Euler

The explicit Euler method is a time-marching finite difference method that aims to calculate the velocity field at each time step from the known velocity from the previous time level. In this method, the time derivative is approximated using a forward difference method while the spatial derivative is approximated using a central difference approach. The numerical equation using the nondimensional governing equation as

$$\frac{U_j^{n+1} - U_j^n}{\Delta\tau} = 1 + \frac{U_{j+1}^n - 2U_j^n + U_{j-1}^n}{(\Delta Y)^2}. \tag{27}$$

To solve for the next time step, U_j^{n+1} is isolated on the left hand side and be calculated directly by

$$U_j^{n+1} = U_j^n + \Delta\tau + \frac{\Delta\tau}{(\Delta Y)^2}\left(U_{j+1}^n - 2U_j^n + U_{j-1}^n\right). \tag{28}$$

The numerical equation can be expressed in matrix form. The spatial grid contains n interior points, which produces an $n \times n$ update matrix. Because all terms on the right-hand side are known at time level n, the velocity vector at time level $n+1$ is obtained through direct matrix multiplication. To simplify the matrix, r is defined to be

$$r = \frac{\Delta\tau}{(\Delta Y)^2} \tag{29}$$

and the Euler update matrix equation form can be equated to

$$\begin{bmatrix} U_1^{n+1} \\ U_2^{n+1} \\ \vdots \\ U_n^{n+1} \end{bmatrix} = \begin{bmatrix} 1-2r & r & 0 & \cdots & 0 \\ r & 1-2r & r & \cdots & 0 \\ 0 & r & 1-2r & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & r \\ 0 & \cdots & 0 & r & 1-2r \end{bmatrix} \begin{bmatrix} U_1^n \\ U_2^n \\ \vdots \\ U_n^n \end{bmatrix} + \Delta\tau \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}. \tag{30}$$

Implicit Euler

Similar to the Explicit Euler method, the implicit method is a time marching finite difference method. However, this method utilizes the velocity field at the next time step to calculate the spatial derivative. In the implicit method, the velocity values at the new time step are unknown and cannot be calculated independently. These values must be determined through solving a system of equations at each time step. The approximations in the implicit method are the same as that in the explicit method; however, the central difference term now incorporates the velocity field at $n+1$.

$$\frac{U_j^{n+1} - U_j^n}{\Delta\tau} = 1 + \frac{U_{j+1}^{n+1} - 2U_j^{n+1} + U_{j-1}^{n+1}}{(\Delta Y)^2} \tag{31}$$

To create the system of equations, all elements that pertain to the time step ahead is moved to the left hand side of the equation as follows

$$U_j^{n+1} - \frac{\Delta\tau}{(\Delta Y)^2}\left(U_{j+1}^{n+1} - 2U_j^{n+1} + U_{j-1}^{n+1}\right) = U_j^n + \Delta\tau \tag{32}$$

Similar to the explicit method, r is defined to simplify the system of equations. Using the spatial approximations, an $n \times n$ tridiagonal matrix is constructed from the velocity values at the interior grid points.

$$\begin{bmatrix} 1+2r & -r & 0 & \cdots & 0 \\ -r & 1+2r & -r & \cdots & 0 \\ 0 & -r & 1+2r & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -r \\ 0 & \cdots & 0 & -r & 1+2r \end{bmatrix} \begin{bmatrix} U_1^{n+1} \\ U_2^{n+1} \\ U_3^{n+1} \\ \vdots \\ U_n^{n+1} \end{bmatrix} = \begin{bmatrix} U_1^n + \Delta\tau \\ U_2^n + \Delta\tau \\ U_3^n + \Delta\tau \\ \vdots \\ U_n^n + \Delta\tau \end{bmatrix}. \tag{33}$$

Fourth-Order Runge-Kutta

An additional method was used to solve for the numerical solution and advance the velocity solution from an initial time to the next time step. This method does this through approximating the slope at four different points within the given time step. These four slopes are then combined using a weighted average to obtain the numerical solution. The governing equation is redefined as $dU/d\tau = F_j(U)$ on the LHS with a central difference method on the RHS.

$$F_j(\mathbf{U}) = 1 + \frac{U_{j+1} - 2U_j + U_{j-1}}{(\Delta Y)^2}. \tag{34}$$

The first Runge-Kutta slope is defined by

$$k_{1,j} = F_j(\mathbf{U}^n). \tag{35}$$

The second being evaluated at a half time step

$$k_{2,j} = F_j\left(\mathbf{U}^n + \frac{\Delta\tau}{2}\mathbf{k}_1\right) \tag{36}$$

The third being

$$k_{3,j} = F_j\left(\mathbf{U}^n + \frac{\Delta\tau}{2}\mathbf{k}_2\right) \tag{37}$$

and the final slope being evaluated at the end of the time step

$$k_{4,j} = F_j\left(\mathbf{U}^n + \Delta\tau\mathbf{k}_3\right). \tag{38}$$

The velocity at the next time step is then found through a weighted average of all the slopes being

$$\mathbf{U}^{n+1} = \mathbf{U}^n + \frac{\Delta\tau}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right) \tag{39}$$

Error and Convergence

The accuracy of the numerical solutions are evaluated by comparing each velocity profile with the exact analytical solution at the same spacial location and final time. The maximum error was calculated for each of the differing numerical methods. The error is defined by

$$E_\infty = \max_j \left| U_j^{\text{numerical}}(\tau_f) - U^{\text{exact}}(Y_j, \tau_f) \right|. \tag{40}$$

To investigate the convergence of each of the methods, each method was performed using multiple values of ΔY and $\Delta\tau$ for a selected r. The grid study utilized the following values of ΔY and r to change the time step.

Table 1: Spatial-grid refinement study with constant r.

Case	ΔY	r	$\Delta\tau = r(\Delta Y)^2$
1	0.200	0.25	0.010000
2	0.100	0.25	0.002500
3	0.050	0.25	0.000625

Table 2: Diffusion-number study with constant ΔY.

Case	ΔY	r	$\Delta\tau = r(\Delta Y)^2$
1	0.100	0.10	0.001000
2	0.100	0.25	0.002500
3	0.100	0.40	0.004000
4	0.100	0.50	0.005000

The convergence rate was calculated by comparing the errors obtained from two grid spacings to determine the order of accuracy.

$$p_Y = \frac{\log(E_2/E_1)}{\log(\Delta Y_2/\Delta Y_1)}. \tag{41}$$

Results and Discussion

The numerical solutions for each of the 3 methods were compared using three grid spacings, $\Delta Y = 0.2, 0.1, 0.05$. The simulation was run at $\tau = 1$ to showcase the fluid flow before it is fully developed. Because the velocity profiles between $\Delta Y = 0.2, 0.05$ bear similar results, these profiles are presented for comparison. All numerical methods follow the analytical velocity profile.

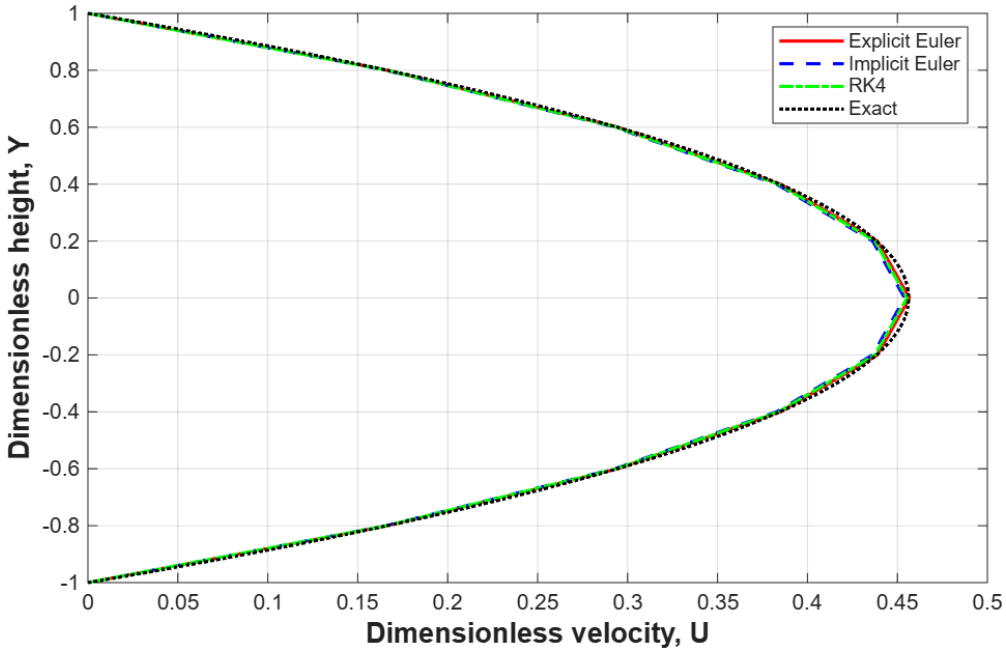


Figure 1: Comparison of numerical and analytical velocity profiles for $\Delta Y = 0.2$.

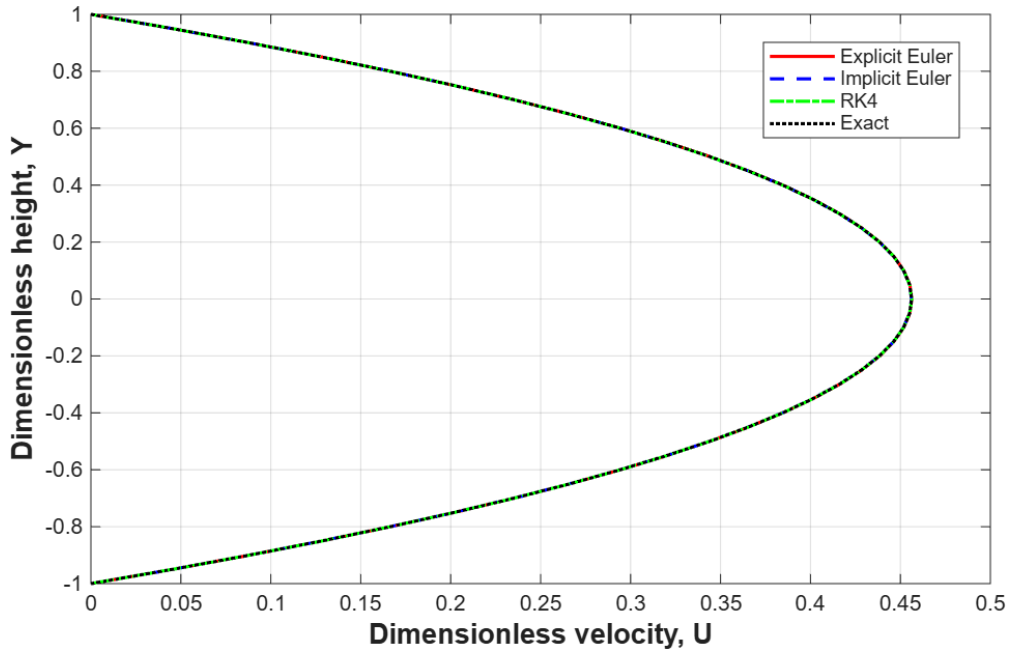


Figure 2: Comparison of numerical and analytical velocity profiles for $\Delta Y = 0.05$.

As showcased in Figures 1 and 2, the velocity profiles of all three methods agree with that of the analytical solution. The visual differences between the numerical solutions and the analytical solution are significantly small for both the grid spacing. There is a noticeable improvement in the profile for $\Delta Y = 0.05$ due to the finer grid sizing, as there appears to be a reduction in truncation error.

The dimensionless flow rate was taken into account through integrating the velocity profile across the channel at each time step. The flow rate from the finest grid spacing is presented, as it provides an accurate representation of the solution.

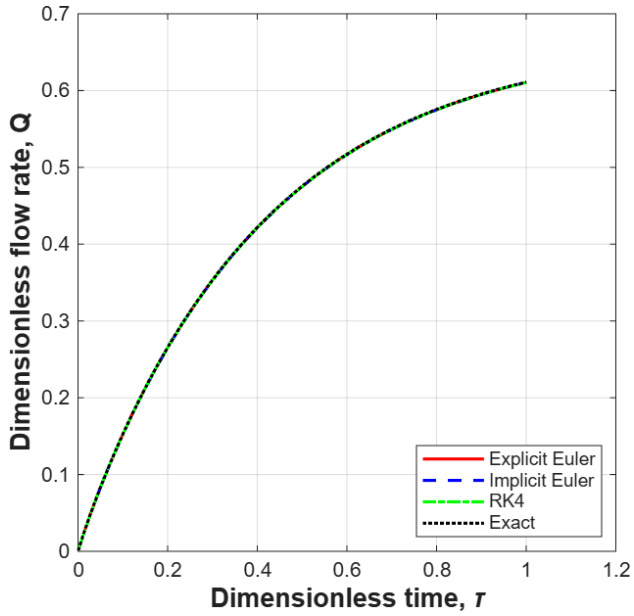


Figure 3: Dimensionless flow rate as a function of dimensionless time for $\Delta Y = 0.05$.

As displayed in Figure 3, the flow rate increases with dimensionless time as the fluid accelerates under the constant pressure gradient and appears to approach steady state. The numerical solutions all closely follow the analytical flow-rate solution with very small visual differences between them. The wall shear is also calculated to observe the velocity gradient at the wall.

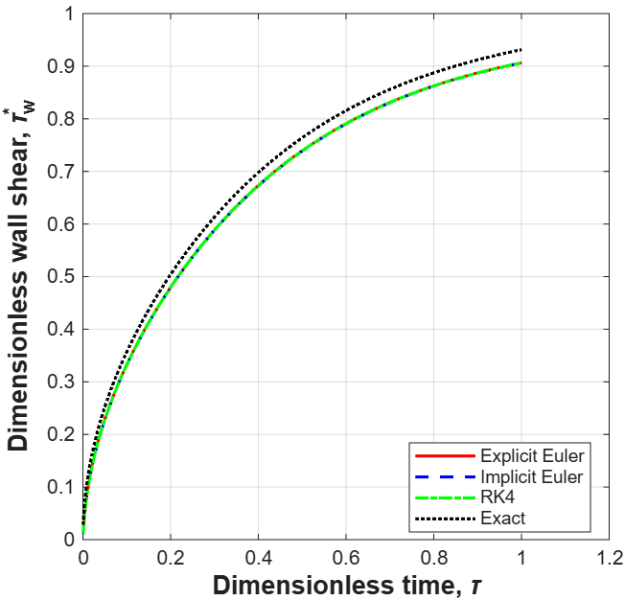


Figure 4: Dimensionless wall shear as a function of dimensionless time for $\Delta Y = 0.05$.

Figure 4 appears to have the greatest difference even at the finest ΔY in the grid study. This phenomenon occurs because the wall shear is calculated from the velocity gradient, and numerical differences can amplify this discretization error. This error can also be attributed from the shear stress being calculated only using 2 points rather than more in areas away from the wall.

The differences between the numerical and analytical solutions were quantified using grid refinement presenting the maximum errors for the velocity profile, flow rate, and wall shear for the selected ΔY values.

Table 3: Maximum errors for various spatial grid spacings with $r = 0.25$.

ΔY	Explicit Euler	Implicit Euler	RK4
	Velocity profile error		
0.200	4.4648×10^{-4}	2.2000×10^{-3}	8.9235×10^{-4}
0.100	3.7987×10^{-4}	2.8439×10^{-4}	4.7637×10^{-5}
0.050	9.5170×10^{-5}	7.1231×10^{-5}	1.1963×10^{-5}
	Flow-rate error		
0.200	5.6000×10^{-3}	9.9000×10^{-3}	7.4000×10^{-3}
0.100	1.4000×10^{-3}	2.5000×10^{-3}	1.8000×10^{-3}
0.050	3.5277×10^{-4}	6.2458×10^{-4}	4.6234×10^{-4}
	Wall-shear error		
0.200	9.8200×10^{-2}	1.0270×10^{-1}	1.0030×10^{-1}
0.100	4.9500×10^{-2}	5.0700×10^{-2}	5.0100×10^{-2}
0.050	2.4900×10^{-2}	2.5200×10^{-2}	2.5000×10^{-2}

Table 3 displays the trend in which error generally decreased as the grid spacing is reduced. The 4th-order Runge-Kutta is observed to have the smallest error for the velocity profile on the finest grid point. Comparing the Euler methods, implicit Euler is slightly more accurate than explicit on the finest grid point. As discussed earlier, wall shear error is relatively high in comparison to that of flow rate and velocity profile due to the discretization error from using the velocity gradient. Nonetheless there is a downward trend, as the grid spacing is reduced when calculating the wall shear stress.

The effects of grid refinement can be seen through a semilog plot of the maximum velocity profile error compared to the grid spacing. The plot showcases the convergence behavior of each of the numerical methods.

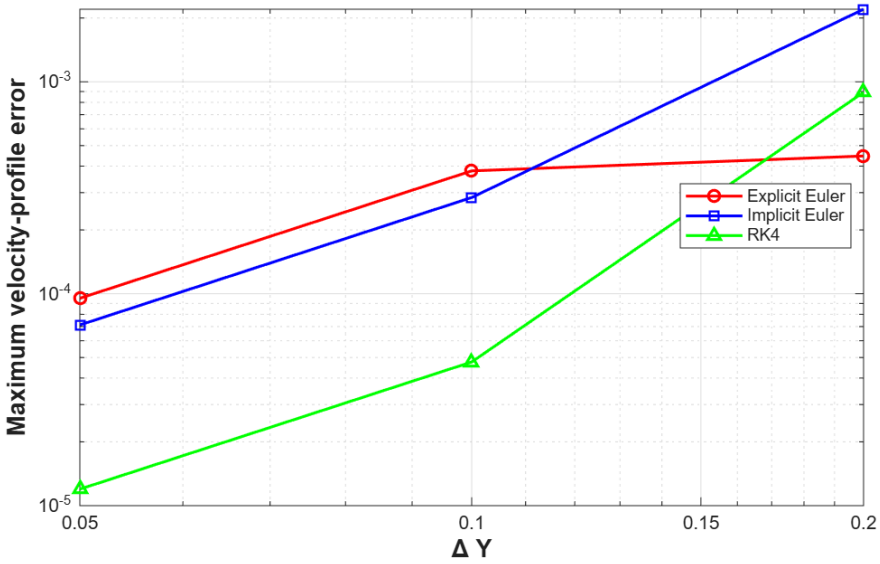


Figure 5: Maximum velocity-profile error as a function of spatial grid spacing.

As seen in Figure 5, the error generally decreases as ΔY is reduced. Finer grid results such as $\Delta Y = 0.1, 0.05$

showcase a clear convergence trend. The course grid of 0.2 does not appear to follow this trend, as explicit Euler appears to have the smallest error. However, this does not indicate that explicit Euler is the most accurate method.

The effect of the diffusion number was then investigated by holding the spatial grid spacing constant at $\Delta Y = 0.1$ while varying r. The time-step sizes were calculated using the equation

$$\Delta\tau = r(\Delta Y)^2.$$

The values of r and $\Delta\tau$ used in this study are listed in Table 2.

Table 4: Maximum errors for different diffusion numbers with $\Delta Y = 0.1$.

r	Explicit Euler	Implicit Euler	RK4
	Velocity profile error		
0.10	8.8933×10^{-5}	3.5561×10^{-4}	2.2229×10^{-4}
0.25	3.8031×10^{-4}	2.8394×10^{-4}	4.8084×10^{-5}
0.40	3.1133×10^{-4}	7.5537×10^{-4}	2.2229×10^{-4}
0.50	4.4648×10^{-4}	8.8855×10^{-4}	2.2229×10^{-4}
	Flow-rate error		
0.10	1.7000×10^{-3}	2.1000×10^{-3}	1.8000×10^{-3}
0.25	1.4000×10^{-3}	2.5000×10^{-3}	1.8000×10^{-3}
0.40	1.2000×10^{-3}	2.9000×10^{-3}	1.8000×10^{-3}
0.50	9.8687×10^{-4}	3.2000×10^{-3}	1.8000×10^{-3}
	Wall-shear error		
0.10	4.9900×10^{-2}	5.0300×10^{-2}	5.0100×10^{-2}
0.25	4.9500×10^{-2}	5.0700×10^{-2}	5.0100×10^{-2}
0.40	4.9200×10^{-2}	5.1100×10^{-2}	5.0100×10^{-2}
0.50	4.9000×10^{-2}	5.1500×10^{-2}	5.0100×10^{-2}

Table 4 shows the effect of varying the diffusion number while maintaining a constant spatial grid spacing of $\Delta Y = 0.1$. The RK4 velocity-profile error remains constnat for most values of r except at $r = 0.25$. The error for both Euler methods appear to increase in value as r increases; however, both methods appear to oscillate in error reduction due to cancellations in spatial and temporal errors. The value $r = 0.5$ was chosen as the max, as the explicit Euler stability requires that

$$r = \frac{\Delta\tau}{(\Delta Y)^2} \leq \frac{1}{2}.$$

The other methods are not restricted by the condition like the explicit method is. Testing values greater than 0.5 resulted in oscillatory and infinitely growing error for the explicit methods. It is implied that as r increase in value for implicit methods, the accuracy would decrease in tandem. The temporal semilog plot visualizes the maximum error discussed by the table above.

An additional diffusion number was utilized to observe the error behaviors of the system. Figure 6 showcases that the RK4 velocity profile error remains constant except at $r = 0.25$, which can be attributed to be an error cancellation between temporal and spatial errors. The plot showcases that both Euler methods appear

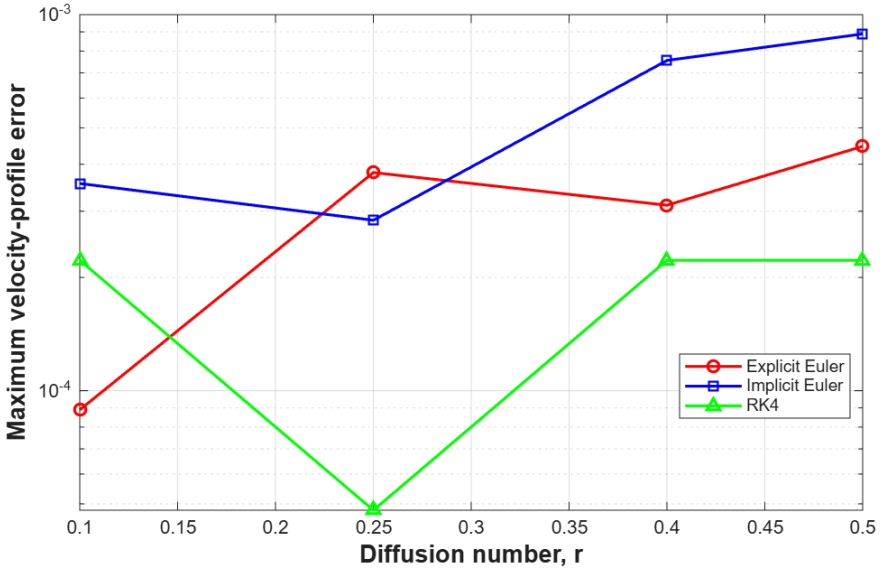


Figure 6: Maximum velocity-profile error as a function of the diffusion number for $\Delta Y = 0.1$.

to have a downward trend when expanded to 4 varying r values. The plot is not perfectly monotonic due to the way spatial and temporal errors interact with each other.

The effect on $\Delta\tau$ was also observed on the 4th-order Runge-Kutta method. The table below showcases the maximum error for each of the differing time steps for a constant ΔY.

Table 5: RK4 errors for different time-step sizes with $\Delta Y = 0.1$.

$\Delta\tau$	r	Velocity error	Flow-rate error	Wall-shear error
0.000625	0.0625	1.5420×10^{-4}	1.8×10^{-3}	5.01×10^{-2}
0.001250	0.1250	8.6671×10^{-5}	1.8×10^{-3}	5.01×10^{-2}
0.002500	0.2500	4.8084×10^{-5}	1.8×10^{-3}	5.01×10^{-2}
0.005000	0.5000	2.2184×10^{-4}	1.8×10^{-3}	5.01×10^{-2}

Table 5 displays that the smallest velocity profile error occurs at $\Delta\tau = 0.0025$. Maximum error does not appear to decrease monotonically when time step is reduced. Errors between flow rate and wall shear stress appear to remain constant as $\Delta\tau$ changes. This behavior can be attributed to the fixed grid spacing possibly limiting the accuracy of the simulation. The figure below further visualizes this behavior.

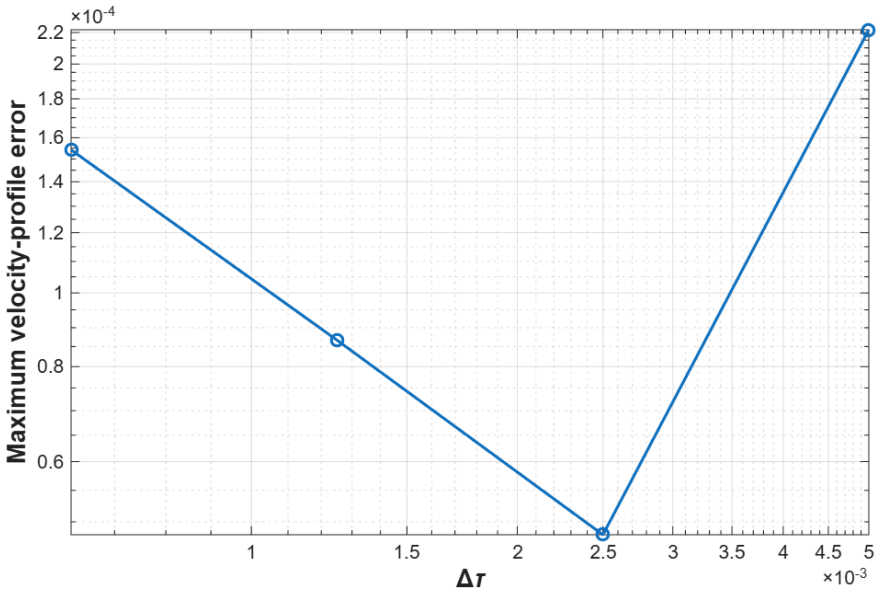


Figure 7: Maximum RK4 velocity-profile error as a function of time-step size for $\Delta Y = 0.1$.

Figure 7 visualizes the table behavior as the error reaches its minimum value at 0.0025. However, the figure does not showcase that a reduction in $\Delta\tau$ results in an error reduction like the two Euler methods show. The non monotonic behavior in the semilog plot might be caused by the fixed grid spacing dominating the temporal error. The sharp decrease in error is likely caused by a partial cancellation between both the temporal and spatial errors.

Conclusion

The developing flow between two parallel plates was investigated using three numerical methods, explicit Euler, implicit Euler, and fourth-order Runge-Kutta methods. All three methods presented velocity profiles that closely agreed with the analytical solution. However, on the finest grid refinement, the implicit Euler method reduced the velocity profile error by 2.3939×10^{-5} in comparison to that of 9.5170×10^{-5} in the explicit Euler method. The fourth-order Runge-Kutta produced the greatest reduction of error across all methods on the finest grid spacing, lowering the error by 8.3207×10^{-5} compared with the explicit Euler method. Overall, reducing the grid spacing reduced the error in both the flow rate and wall shear stress as well. However, while wall shear stress did decrease with grid refinement, its error did remain relatively large in comparison to the velocity and flow rate. This is mainly due to how wall shear was calculated by numerically differentiating the velocity profile at the wall, which in turn amplified the discretization error. A better alternative to solving for shear would be using a higher order approximation that utilizes more points along the wall.

The effects of the diffusion number was also investigated while maintaining a constant grid spacing. Similar to the trends of changing the spatial grid spacing, both Euler methods saw an overall decrease as grid spacing is reduced. A key difference was noted in the behavior of the RK4, as error remained mostly constant aside from at $r = 0.25$ where there appears to be a cancellation between the temporal and spatial errors. While the general trend of decreasing $\Delta\tau$ shows a decrease in the Euler method errors, this may only be true up to a certain point. When the temporal error becomes small enough, the spatial discretization error may dominate the total error in the solution. Reducing time step does not always result in a proportional reduction in total error.

Overall, the results of the study showcases that numerical accuracy can depend on both spatial and temporal resolutions. It is necessary to refine both grids, as spatial error may dominate any temporal error stopping any further reductions in a numerical solution. Between all 3 methods, RK4 produces the most accurate results on finer grid spacing due to its methods in evaluating multiple slopes within each time step.

Handwritten Derivations

Nondimensionalization

The dimensional governing equation is

$$\frac{\partial u}{\partial t} = -\frac{1}{\rho}\frac{dp}{dx} + \nu\frac{\partial^2 u}{\partial y^2}. \tag{42}$$

The dimensionless variables are defined as

$$U = \frac{u}{U_\infty}, \qquad Y = \frac{y}{h}, \qquad \tau = \frac{t}{t_c}, \tag{43}$$

where the characteristic time is selected as

$$t_c = \frac{h^2}{\nu}. \tag{44}$$

$$\tau = \frac{\nu t}{h^2}. \tag{45}$$

The pressure-gradient term is represented by

$$G = -\frac{1}{\rho}\frac{dp}{dx}. \tag{46}$$

$$u = U_\infty U, \qquad y = hY, \qquad t = \frac{h^2}{\nu}\tau, \tag{47}$$

$$\frac{\partial U}{\partial \tau} = \frac{Gh^2}{\nu U_\infty} + \frac{\partial^2 U}{\partial Y^2}. \tag{48}$$

The characteristic velocity is selected as

$$U_\infty = \frac{Gh^2}{\nu}. \tag{49}$$

$$\frac{Gh^2}{\nu U_\infty} = 1, \tag{50}$$

and the dimensionless governing equation is

$$\boxed{\frac{\partial U}{\partial \tau} = 1 + \frac{\partial^2 U}{\partial Y^2}}. \tag{51}$$

The boundary conditions are

$$U(-1, \tau) = 0, \qquad U(1, \tau) = 0, \tag{52}$$

and the initial condition is

$$U(Y, 0) = 0. \tag{53}$$

Analytical Solution

The velocity field is separated into steady and transient components:

$$U(Y,\tau) = U_s(Y) + V(Y,\tau). \tag{54}$$

For the steady-state solution,

$$\frac{\partial U_s}{\partial \tau} = 0. \tag{55}$$

$$0 = 1 + \frac{d^2U_s}{dY^2}. \tag{56}$$

$$\frac{dU_s}{dY} = -Y + C_1. \tag{57}$$

$$U_s(Y) = -\frac{Y^2}{2} + C_1Y + C_2. \tag{58}$$

Applying the boundary conditions

$$U_s(-1) = 0, \qquad U_s(1) = 0. \tag{59}$$

$$0 = -\frac{(-1)^2}{2} + C_1(-1) + C_2, \tag{60}$$

$$0 = -\frac{1}{2} - C_1 + C_2. \tag{61}$$

$$0 = -\frac{(1)^2}{2} + C_1(1) + C_2, \tag{62}$$

$$0 = -\frac{1}{2} + C_1 + C_2. \tag{63}$$

Adding the two equations gives

$$0 = -1 + 2C_2, \tag{64}$$

so

$$C_2 = \frac{1}{2}. \tag{65}$$

Subtracting the first equation from the second gives

$$0 = 2C_1, \tag{66}$$

so

$$C_1 = 0. \tag{67}$$

Therefore, the steady-state velocity profile is

$$\boxed{U_s(Y) = \frac{1}{2}(1 - Y^2)}. \tag{68}$$

The transient component satisfies

$$\frac{\partial V}{\partial \tau} = \frac{\partial^2 V}{\partial Y^2}. \tag{69}$$

The boundary conditions for the transient component are obtained from the boundary conditions for the total velocity:

$$U(-1,\tau) = 0, \qquad U(1,\tau) = 0. \tag{70}$$

Since

$$U(Y,\tau) = U_s(Y) + V(Y,\tau) \tag{71}$$

and

$$U_s(-1) = U_s(1) = 0, \tag{72}$$

$$V(Y,0) = -\frac{1}{2}(1-Y^2). \tag{73}$$

$$U(Y,\tau) = U_s(Y) + V(Y,\tau). \tag{74}$$

$$V(Y,\tau) = U(Y,\tau) - U_s(Y). \tag{75}$$

$$\frac{\partial}{\partial \tau}\left(U_s + V\right) = 1 + \frac{\partial^2}{\partial Y^2}\left(U_s + V\right). \tag{76}$$

$$\frac{\partial U_s}{\partial \tau} + \frac{\partial V}{\partial \tau} = 1 + \frac{d^2 U_s}{dY^2} + \frac{\partial^2 V}{\partial Y^2}. \tag{77}$$

$$\frac{\partial U_s}{\partial \tau} = 0. \tag{78}$$

$$0 = 1 + \frac{d^2 U_s}{dY^2}. \tag{79}$$

$$\frac{\partial V}{\partial \tau} = \frac{\partial^2 V}{\partial Y^2}. \tag{80}$$

$$V(Y,\tau) = F(Y)T(\tau). \tag{81}$$

$$\frac{\partial}{\partial \tau}\left[F(Y)T(\tau)\right] = \frac{\partial^2}{\partial Y^2}\left[F(Y)T(\tau)\right]. \tag{82}$$

$$F(Y)\frac{dT}{d\tau} = T(\tau)\frac{d^2 F}{dY^2}. \tag{83}$$

Dividing both sides by $F(Y)T(\tau)$ gives

$$\frac{1}{T}\frac{dT}{d\tau} = \frac{1}{F}\frac{d^2 F}{dY^2}. \tag{84}$$

$$\frac{1}{T}\frac{dT}{d\tau} = \frac{1}{F}\frac{d^2 F}{dY^2} = -\lambda. \tag{85}$$

This produces two ordinary differential equations:

$$\frac{dT}{d\tau} + \lambda T = 0 \tag{86}$$

and

$$\frac{d^2F}{dY^2} + \lambda F = 0. \tag{87}$$

$$\frac{dT}{d\tau} = -\lambda T. \tag{88}$$

$$\frac{1}{T}\frac{dT}{d\tau} = -\lambda. \tag{89}$$

Integrating both sides with respect to τ gives

$$\int \frac{1}{T}\, dT = -\lambda \int d\tau. \tag{90}$$

$$\ln|T| = -\lambda\tau + C. \tag{91}$$

$$T(\tau) = Ce^{-\lambda\tau}. \tag{92}$$

$$T(\tau) = e^{-\lambda\tau}. \tag{93}$$

$$\lambda = k^2. \tag{94}$$

$$\frac{d^2F}{dY^2} + k^2F = 0. \tag{95}$$

$$F(Y) = A\cos(kY) + B\sin(kY). \tag{96}$$

Applying the boundary conditions

$$F(1) = A\cos(k) + B\sin(k) = 0. \tag{97}$$

$$F(-1) = A\cos(-k) + B\sin(-k) = 0. \tag{98}$$

$$\cos(-k) = \cos(k), \qquad \sin(-k) = -\sin(k), \tag{99}$$

$$A\cos(k) - B\sin(k) = 0. \tag{100}$$

$$A\cos(k) + B\sin(k) = 0 \tag{101}$$

and

$$A\cos(k) - B\sin(k) = 0. \tag{102}$$

Adding

$$2A\cos(k) = 0. \tag{103}$$

Subtracting

$$2B\sin(k) = 0. \tag{104}$$

$$B = 0. \tag{105}$$

$$A\cos(k) = 0. \tag{106}$$

$$\cos(k) = 0. \tag{107}$$

$$k_n = \frac{(2n+1)\pi}{2}, \qquad n = 0, 1, 2, \ldots \tag{108}$$

and

$$\lambda_n = k_n^2 = \frac{(2n+1)^2\pi^2}{4}. \tag{109}$$

$$F_n(Y) = \cos\left(\frac{(2n+1)\pi Y}{2}\right). \tag{110}$$

$$T_n(\tau) = \exp\left[-\frac{(2n+1)^2\pi^2}{4}\tau\right]. \tag{111}$$

$$V(Y,\tau) = \sum_{n=0}^{\infty} A_n \cos\left(\frac{(2n+1)\pi Y}{2}\right) \exp\left[-\frac{(2n+1)^2\pi^2}{4}\tau\right]. \tag{112}$$

At $\tau = 0$, the transient solution becomes

$$V(Y,0) = \sum_{n=0}^{\infty} A_n \cos\left(\frac{(2n+1)\pi Y}{2}\right). \tag{113}$$

$$A_n = \int_{-1}^{1} V(Y,0)\cos\left(\frac{(2n+1)\pi Y}{2}\right) dY. \tag{114}$$

$$A_n = -\frac{1}{2}\int_{-1}^{1} (1-Y^2)\cos(k_n Y)\, dY. \tag{115}$$

$$A_n = -\frac{1}{2}\left[\int_{-1}^{1} \cos(k_n Y)\, dY - \int_{-1}^{1} Y^2\cos(k_n Y)\, dY\right]. \tag{116}$$

$$\int_{-1}^{1} \cos(k_n Y)\, dY = \left[\frac{\sin(k_n Y)}{k_n}\right]_{-1}^{1}. \tag{117}$$

$$\int_{-1}^{1} \cos(k_n Y)\, dY = \frac{\sin(k_n) - \sin(-k_n)}{k_n}. \tag{118}$$

Since $\sin(-k_n) = -\sin(k_n)$,

$$\int_{-1}^{1} \cos(k_n Y)\, dY = \frac{2\sin(k_n)}{k_n}. \tag{119}$$

$$\int Y^2\cos(k_n Y)\, dY = \frac{Y^2\sin(k_n Y)}{k_n} - \frac{2}{k_n}\int Y\sin(k_n Y)\, dY. \tag{120}$$

$$\int Y \sin(k_n Y)\, dY = -\frac{Y\cos(k_n Y)}{k_n} + \frac{\sin(k_n Y)}{k_n^2}. \tag{121}$$

$$\int Y^2 \cos(k_n Y)\, dY = \frac{Y^2 \sin(k_n Y)}{k_n} + \frac{2Y\cos(k_n Y)}{k_n^2} - \frac{2\sin(k_n Y)}{k_n^3}. \tag{122}$$

$$\int_{-1}^{1} Y^2 \cos(k_n Y)\, dY = \frac{2\sin(k_n)}{k_n} + \frac{4\cos(k_n)}{k_n^2} - \frac{4\sin(k_n)}{k_n^3}. \tag{123}$$

$$\int_{-1}^{1} Y^2 \cos(k_n Y)\, dY = \frac{2\sin(k_n)}{k_n} - \frac{4\sin(k_n)}{k_n^3}. \tag{124}$$

$$A_n = -\frac{1}{2}\left[\frac{2\sin(k_n)}{k_n} - \left(\frac{2\sin(k_n)}{k_n} - \frac{4\sin(k_n)}{k_n^3}\right)\right]. \tag{125}$$

$$A_n = -\frac{1}{2}\left[\frac{4\sin(k_n)}{k_n^3}\right]. \tag{126}$$

$$A_n = -\frac{2\sin(k_n)}{k_n^3}. \tag{127}$$

$$k_n = \frac{(2n+1)\pi}{2}, \tag{128}$$

$$\sin(k_n) = (-1)^n. \tag{129}$$

$$k_n^3 = \left(\frac{(2n+1)\pi}{2}\right)^3 = \frac{(2n+1)^3\pi^3}{8}. \tag{130}$$

$$A_n = -\frac{2(-1)^n}{(2n+1)^3\pi^3/8}. \tag{131}$$

$$\boxed{A_n = -\frac{16(-1)^n}{(2n+1)^3\pi^3}}. \tag{132}$$

$$V(Y,\tau) = -\frac{16}{\pi^3}\sum_{n=0}^{\infty}\frac{(-1)^n}{(2n+1)^3}\cos\left(\frac{(2n+1)\pi Y}{2}\right)\exp\left[-\frac{(2n+1)^2\pi^2}{4}\tau\right]. \tag{133}$$

Combining the Steady state solution and transient solution gives

$$\boxed{U(Y,\tau) = \frac{1}{2}(1-Y^2) - \frac{16}{\pi^3}\sum_{n=0}^{\infty}\frac{(-1)^n}{(2n+1)^3}\cos\left(\frac{(2n+1)\pi Y}{2}\right)\exp\left[-\frac{(2n+1)^2\pi^2}{4}\tau\right]}. \tag{134}$$

Explicit Euler Method

$$\frac{\partial U}{\partial \tau} \approx \frac{U_j^{n+1} - U_j^n}{\Delta \tau} \tag{135}$$

and

$$\frac{\partial^2 U}{\partial Y^2} \approx \frac{U_{j+1}^n - 2U_j^n + U_{j-1}^n}{(\Delta Y)^2}. \tag{136}$$

$$\frac{U_j^{n+1} - U_j^n}{\Delta \tau} = 1 + \frac{U_{j+1}^n - 2U_j^n + U_{j-1}^n}{(\Delta Y)^2}. \tag{137}$$

$$r = \frac{\Delta \tau}{(\Delta Y)^2}. \tag{138}$$

$$U_j^{n+1} - U_j^n = \Delta \tau + r\left(U_{j+1}^n - 2U_j^n + U_{j-1}^n\right). \tag{139}$$

$$\boxed{U_j^{n+1} = rU_{j-1}^n + (1-2r)U_j^n + rU_{j+1}^n + \Delta \tau}. \tag{140}$$

$$\mathbf{U}^{n+1} = \mathbf{A}_E \mathbf{U}^n + \Delta \tau \mathbf{b}, \tag{141}$$

$$\mathbf{A}_E = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ r & 1-2r & r & \cdots & 0 \\ 0 & r & 1-2r & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & r \\ 0 & \cdots & 0 & r & 1 \end{bmatrix}. \tag{142}$$

$$\mathbf{b} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ \vdots \\ 1 \\ 0 \end{bmatrix}. \tag{143}$$

Implicit Euler

$$\frac{\partial U}{\partial \tau} \approx \frac{U_j^{n+1} - U_j^n}{\Delta \tau} \tag{144}$$

and

$$\frac{\partial^2 U}{\partial Y^2} \approx \frac{U_{j+1}^{n+1} - 2U_j^{n+1} + U_{j-1}^{n+1}}{(\Delta Y)^2}. \tag{145}$$

$$\frac{U_j^{n+1} - U_j^n}{\Delta \tau} = 1 + \frac{U_{j+1}^{n+1} - 2U_j^{n+1} + U_{j-1}^{n+1}}{(\Delta Y)^2}. \tag{146}$$

$$U_j^{n+1} - U_j^n = \Delta \tau + r\left(U_{j+1}^{n+1} - 2U_j^{n+1} + U_{j-1}^{n+1}\right). \tag{147}$$

$$U_j^{n+1} - U_j^n = \Delta \tau + rU_{j+1}^{n+1} - 2rU_j^{n+1} + rU_{j-1}^{n+1}. \tag{148}$$

$$-rU_{j-1}^{n+1} + (1+2r)U_j^{n+1} - rU_{j+1}^{n+1} = U_j^n + \Delta\tau. \tag{149}$$

$$\mathbf{A}_I \mathbf{U}^{n+1} = \mathbf{U}^n + \Delta\tau \mathbf{b}, \tag{150}$$

$$\mathbf{A}_I = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ -r & 1+2r & -r & \cdots & 0 \\ 0 & -r & 1+2r & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -r \\ 0 & \cdots & 0 & -r & 1 \end{bmatrix}. \tag{151}$$

$$\mathbf{U}^n + \Delta\tau \mathbf{b}, \tag{152}$$

$$\mathbf{b} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ \vdots \\ 1 \\ 0 \end{bmatrix}. \tag{153}$$

$$\boxed{\mathbf{U}^{n+1} = \mathbf{A}_I^{-1} \left(\mathbf{U}^n + \Delta\tau \mathbf{b} \right)} \tag{154}$$

MATLAB Code: Initial w/ some errors

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Midterm - Fluid Flow between 2 Parallel Plates
% By: Hannah Douthit
% AE 601
% Initial Code with bugs and errors
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1. Scaling and Equations
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% du/dt = -1/rho * dP/dx + nu * d^2u/dy^2
% dU/dtau = G + d^2U/dY^2

clear; clc;
h =1;

%change spacing
dy = 0.1;        %grid spacing; change
dt = 0.001;        %time spacing change
t = 10;

%nondim
Y = -h:dy:h;
tau = 0:dt:t;

%bc
ubot = 0;
utop = 0;

%ic
U_i = zeros(size(Y));


%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2. Exact Solution
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Uss = -Y.^2 /2 + 1/2;
%
% N = 500;
% tsum = zeros(size(Y));
%
% for n = 0:N
%     m = 2*n +1;
%
%     term = ((-1)^n/m^3) .* cos(m*pi*Y/2)    .* exp(-(m^2*pi^2/4)*tau);
%     tsum = tsum + term;
% end
%
```

```
% Utrans = -16/pi^3 *  tsum;
%
% U_ex = Uss + Utrans;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3. Numerical Solutions
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

r = dt / (dy)^2;
Q = [];

% i. Explicit

N = length(Y);
Uta = zeros(N, 1);
Utn = U_i';
ones = ones(N,1);

%matrix
LHS = r;
M = 1-2*r;
RHS = r;
A = zeros(N,N);

for i = 1:N
    A(i,i) = M;
    if i > 1
        A(i, i-1) = LHS;
    end

    if i < N
        A(i, i+1) = RHS;
    end
end

tstart = 0;
tend = 10;
count = tstart;
tau_hist = [];
ws = [];

while count <= tend
    Uta = A*Utn + dt*ones;
    count = count + dt;
    Utn = Uta;
    %flow rate
    tau_hist(end+1) = count + dt;
    Q(end+1) = trapz(Y,Uta);
    %wall shear
```

```
    ws(end+1) = Uta(1)/dy;


end



% ii. Implicit

% LHSi = -r;
% Mi = 1 + 2*r;
% RHSi = -r;
% A_i= zeros(N,N);
%
% for i = 1:N
%     A_i(i,i) = M;
%     if i > 1
%         A_i(i, i-1) = LHS;
%     end
%
%     if i < N
%         A_i(i, i+1) = RHS;
%     end
% end
%
% Uta_i = zeros(N,1);
% Utn_i = U_i';
% ones = ones(N,1);
%
% count = tstart;
%
% while count <= tend
%     Uta_i = (Utn_i + dt*ones)/ A_i;
%     count = count + dt;
%     Utn_i = Uta_i;
% end



%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4. Results
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Velocity Profile
% Explicit
figure;
plot(Uta, Y, 'LineWidth',1.5)
xlabel('velocity [m/s]', 'FontSize', 13, 'FontWeight', 'bold')
ylabel('Height [m]', FontSize= 13, FontWeight= 'bold')
```

```
grid on;
pbaspect([ 1 1.5 1])

% Implicit

%Flow Rate over time
% Explicit
figure;
plot(tau, Q, 'LineWidth',1.5)
xlabel('tau', 'FontSize', 13, 'FontWeight', 'bold')
ylabel('Flow Rate Q', FontSize= 13, FontWeight= 'bold')
grid on;
pbaspect( [1 1 1])

% Implicit

% Wall Shear
%   Explicit
figure;
plot(tau, ws, 'LineWidth',1.5)
xlabel('tau', 'FontSize', 13, 'FontWeight', 'bold')
ylabel('Wall Shear \tau', FontSize= 13, FontWeight= 'bold')
grid on;
pbaspect( [1 1 1])

% Implicit


%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4. 4th- Order Runge Kutta
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

L = zeros(N,N);
for i = 1:N
    L(i,i) = -2;
    if i > 1
        L(i, i-1) = 1;
    end

    if i < N
        L(i, i+1) = 1;
    end
end

L = L/dy^2;

N = size(L,1);

source = ones(N,1);
```

```
F = @(U) source + L*U(:);

Utnrk = U_i(:);

count = tstart;

N = length(Y);
Utark = zeros(N, 1);
Utnrk = U_i';
Qr = [];
wsr = [];

while count <= tend
    k1 = F(Utnrk);
    Uhalf = Utnrk + (dt/2)*k1;

    k2 = F(Uhalf);
    Uqua = Utnrk + (dt/2)*k2;

    k3 = F(Uqua);
    Ufull = Utnrk + dt*k3;

    k4 = F(Ufull);

    Utark = Utnrk + (dt/6)*(k1 + 2*k2 + 2*k3 + k4);
    Utnrk = Utark;
    count = count + dt;

    %flow
    Qr(end+1) = trapz(Y,Utark);
    %wall shear
    wsr(end+1) = Utark(1)/dy;

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5. Error Compariosn
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% max error for each method
Ee= max(abs(Uta - U_ex));
Ei = max(abs(Uta_i - U_ex));
Erk = max(abs(Utark - U_ex));
```

Final Matlab Code

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Midterm - Fluid Flow between 2 Parallel Plates
% By: Hannah Douthit
% AE 601
%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1. Scaling and Equations
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% du/dt = -1/rho * dP/dx + nu * d^2u/dy^2
% dU/dtau = G + d^2U/dY^2

clear; clc;
h =1;

%change spacing
dy = 0.1;        %grid spacing; change 0.2 0.1 0.05
% r = 0.25;     % change r value with constant grid spacing
% dt = r*dy^2;
dt = 0.000625;     %change for each run rk4
t = 1;

%nondim
Y = -h:dy:h;
tau = 0:dt:t;

%bc
ubot = 0;
utop = 0;

%ic
U_i = zeros(size(Y));


%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2. Exact Solution
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

tau_final = tau(end);
Yex = linspace(-1,1,1000);
Uss = -Yex.^2/2 + 1/2;

N = 500;
tsum = zeros(size(Yex));
```

```
for n = 0:N

    m = 2*n + 1;

    term = ((-1)^n/m^3) .* ...
        cos(m*pi*Yex/2) .* ...
        exp(-(m^2*pi^2/4)*tau_final);

    tsum = tsum + term;

end

Utrans = -16/pi^3 * tsum;

U_ex = Uss + Utrans;


%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3. Numerical Solutions
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

r = dt / (dy)^2;
Q = [];

% i. Explicit

N = length(Y);
Uta = zeros(N, 1);
Utn = U_i';
forcing = zeros(N,1);
forcing(2:N-1) = 1;

%matrix
LHS = r;
M = 1-2*r;
RHS = r;
A = zeros(N,N);

%boundary
A(1,1) = 1;
A(N,N) = 1;

for i = 2:N-1
    A(i,i) = M;
    if i > 1
        A(i, i-1) = LHS;
    end

    if i < N
```

```
        A(i, i+1) = RHS;
    end
end

%initialize
tstart = 0;
tend = 1;
count = tstart;
tau_hist = [];
ws = [];

%loop
while count < tend
    Uta = A*Utn + dt*forcing;
    count = count + dt;
    Utn = Uta;
    %flow rate
    tau_hist(end+1) = count;
    Q(end+1) = trapz(Y,Uta);
    %wall shear
    ws(end+1) = (Uta(2)-Uta(1))/dy;
end


% ii. Implicit
% matrix
LHSi = -r;
Mi = 1 + 2*r;
RHSi = -r;

A_i = zeros(N,N);

A_i(1,1) = 1;
A_i(N,N) = 1;

for i = 2:N-1
    A_i(i,i) = Mi;
    A_i(i,i-1) = LHSi;
    A_i(i,i+1) = RHSi;
end

%initialize while loop
forcing_i = zeros(N,1);
forcing_i(2:N-1) = 1;

Uta_i = zeros(N,1);
Utn_i = U_i';
Q_i = [];
ws_i = [];
```

```
tau_hist_i = [];

count = tstart;

%loop
while count < tend

    rhs_i = Utn_i + dt*forcing_i;
    Uta_i = A_i \ rhs_i;

    count = count + dt;
    Utn_i = Uta_i;

    %flow rate
    tau_hist_i(end+1) = count;
    Q_i(end+1) = trapz(Y, Uta_i);

    %shear stress
    ws_i(end+1) = (Uta_i(2)-Uta_i(1))/dy;
end



%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4. 4th- Order Runge Kutta
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%matrix
N = length(Y);
L = zeros(N,N);

for i = 2:N-1
    L(i,i) = -2;
    L(i,i-1) = 1;
    L(i,i+1) = 1;
end

L = L/dy^2;

%initialize
source = zeros(N,1);
source(2:N-1) = 1;

% loop solving for each of the four point slops
F = @(U) source + L*U;

Utnrk = U_i';
Utark = zeros(N,1);
```

```
Qr = [];
wsr = [];
tau_hist_r = [];

count = tstart;

while count < tend

    k1 = F(Utnrk);

    Uhalf = Utnrk + (dt/2)*k1;
    k2 = F(Uhalf);

    Uqua = Utnrk + (dt/2)*k2;
    k3 = F(Uqua);

    Ufull = Utnrk + dt*k3;
    k4 = F(Ufull);

    Utark = Utnrk + (dt/6)*(k1 + 2*k2 + 2*k3 + k4);

    Utnrk = Utark;
    count = count + dt;

    tau_hist_r(end+1) = count;
    Qr(end+1) = trapz(Y,Utark);
    wsr(end+1) = (Utark(2)-Utark(1))/dy;

end


%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5. Results
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%


%using tau_his to get Qex and Tauex for plotting purposes
Q_exact = zeros(size(tau_hist));
ws_exact = zeros(size(tau_hist));

Nterms = 500;

for k = 1:length(tau_hist)

    tau_k = tau_hist(k);
    sum_Q = 0;
    sum_ws = 0;

    for n = 0:Nterms
```

```
        m = 2*n + 1;

        sum_Q = sum_Q + (1/m^4) * ...
            exp(-(m^2*pi^2/4)*tau_k);

        sum_ws = sum_ws + (1/m^2) * ...
            exp(-(m^2*pi^2/4)*tau_k);

    end

    Q_exact(k) = 2/3 - (64/pi^4)*sum_Q;
    ws_exact(k) = 1 - (8/pi^2)*sum_ws;

end


% Velocity Profile

plot(Uta,Y,'r-','LineWidth',1.5)
hold on
plot(Uta_i,Y,'b--','LineWidth',1.5)
plot(Utark,Y,'g-.','LineWidth',1.5)
plot(U_ex,Yex,'k:','LineWidth',1.5)
grid on
xlabel('Dimensionless velocity, U','FontSize',13,'FontWeight','bold')
ylabel('Dimensionless height, Y','FontSize',13,'FontWeight','bold')
legend('Explicit Euler','Implicit Euler','RK4','Exact', ...
    'Location','best')
xlim([0 0.5])


% Flow Rate over Time
figure
plot(tau_hist,Q,'r-','LineWidth',1.5)
hold on
plot(tau_hist_i,Q_i,'b--','LineWidth',1.5)
plot(tau_hist_r,Qr,'g-.','LineWidth',1.5)
plot(tau_hist,Q_exact,'k:','LineWidth',1.5)

xlabel('Dimensionless time, \tau','FontSize',13,'FontWeight','bold')
ylabel('Dimensionless flow rate, Q','FontSize',13,'FontWeight','bold')

legend('Explicit Euler','Implicit Euler','RK4','Exact', ...
    'Location','best')
grid on
pbaspect([1 1 1])
```

```
% Wall Shear over Time
figure
plot(tau_hist,ws,'r-','LineWidth',1.5)
hold on
plot(tau_hist_i,ws_i,'b--','LineWidth',1.5)
plot(tau_hist_r,wsr,'g-.','LineWidth',1.5)
plot(tau_hist,ws_exact,'k:','LineWidth',1.5)

xlabel('Dimensionless time, \tau','FontSize',13,'FontWeight','bold')
ylabel('Dimensionless wall shear, \tau_w^*','FontSize',13,'FontWeight','bold')

legend('Explicit Euler','Implicit Euler','RK4','Exact', ...
    'Location','best')

grid on
pbaspect([1 1 1])


%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5. Error Comparison
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
U_ex_num = interp1(Yex(:),U_ex(:),Y(:));
% velocity
Ee = max(abs(Uta(:) - U_ex_num(:)));
Ei = max(abs(Uta_i(:) - U_ex_num(:)));
Erk = max(abs(Utark(:) - U_ex_num(:)));
% flow rate
Ee_Q = max(abs(Q - Q_exact));
Ei_Q = max(abs(Q_i - Q_exact));
Erk_Q = max(abs(Qr - Q_exact));

% wall shear
Ee_ws = max(abs(ws - ws_exact));
Ei_ws = max(abs(ws_i - ws_exact));
Erk_ws = max(abs(wsr - ws_exact));

% Error plot
Uta = Uta(:);
Uta_i = Uta_i(:);
Utark = Utark(:);
U_ex_num = U_ex_num(:);

error_explicit = abs(Uta-U_ex_num);
error_implicit = abs(Uta_i-U_ex_num);
error_RK4 = abs(Utark-U_ex_num);

figure
plot(Y(:),error_explicit,'r-','LineWidth',1.5)
hold on
```

```
plot(Y(:),error_implicit,'b--','LineWidth',1.5)
plot(Y(:),error_RK4,'g-.','LineWidth',1.5)
hold off

xlabel('Y')
ylabel('Absolute error')
legend('Explicit Euler','Implicit Euler','RK4')
grid on

% grid study values
% spatial loglog plot
dy_values = [0.05, 0.1, 0.2];

error_explicit_grid = [9.5170e-05, 3.7987e-04, 4.4648e-04];
error_implicit_grid = [7.1231e-05, 2.8439e-04, 2.2000e-03];
error_RK4_grid = [1.1963e-05, 4.7637e-05, 8.9235e-04];


figure
loglog(dy_values,error_explicit_grid,'r-o','LineWidth',1.5)
hold on
loglog(dy_values,error_implicit_grid,'b-s','LineWidth',1.5)
loglog(dy_values,error_RK4_grid,'g-^','LineWidth',1.5)

xlabel('\Delta Y','FontSize',13,'FontWeight','bold')
ylabel('Maximum velocity-profile error','FontSize',13,'FontWeight','bold')

legend('Explicit Euler','Implicit Euler','RK4', ...
    'Location','best')
grid on

% Temporal grid-spacing study by changing r

r_values = [0.10, 0.25, 0.40, 0.50];

error_explicit_r = [8.8933e-05, 3.8031e-04, ...
                    3.1133e-04, 4.4648e-04];

error_implicit_r = [3.5561e-04, 2.8394e-04, ...
                    7.5537e-04, 8.8855e-04];

error_RK4_r = [2.2229e-04, 4.8084e-05, ...
               2.2229e-04, 2.2229e-04];

figure
semilogy(r_values,error_explicit_r,'r-o','LineWidth',1.5)
hold on
semilogy(r_values,error_implicit_r,'b-s','LineWidth',1.5)
semilogy(r_values,error_RK4_r,'g-^','LineWidth',1.5)
```

```
xlabel('Diffusion number, r','FontSize',13,'FontWeight','bold')
ylabel('Maximum velocity-profile error','FontSize',13,'FontWeight','bold')

legend('Explicit Euler','Implicit Euler','RK4','Location','best')
grid on

%RK4 study
dt_values = [0.005, 0.0025, 0.00125, 0.000625];

RK4_dt_errors = [2.2184e-04, 4.8084e-05, ...
    8.6671e-05, 1.5420e-04];

figure
loglog(dt_values,RK4_dt_errors,'-o','LineWidth',1.5)

xlabel('\Delta\tau','FontSize',13,'FontWeight','bold')
ylabel('Maximum velocity-profile error','FontSize',13,'FontWeight','bold')

grid on
```